

C++ STL — std::string Cheat Sheet

DSA • Competitive Programming • Interviews | Standalone quick reference

Mental model: std::string is a C++ string object representing a sequence of characters. It has similarities to a contiguous vector<char>, but it is **not literally a vector<char>**; it provides string-specific operations.

1. Declaration / Initialization

```
#include <string>
using namespace std;

string s;
string s1 = "HELLO";
string s2("HELLO");
string s3 = "";
```

2. Access & Basic Information

Operation	Example	Meaning	Typical cost
Index	s[i]	Character at index i	O(1)
Size	s.size()	Number of characters	O(1)
Length	s.length()	Same result as size()	O(1)
Empty	s.empty()	Whether size is 0	O(1)
First	s.front()	First character	O(1)
Last	s.back()	Last character	O(1)

Index rule: if size is n, valid indices are 0 through n-1. Last character = s[n-1] or s.back(). s[i] does not bounds-check; s.at(i) does.

3. Traversal

```
for (int i = 0; i < (int)s.size(); ++i) cout << s[i];
```

```
for (char c : s) cout << c;
```

```
for (char& c : s) { /* modify c */ }
```

```
for (auto it = s.begin(); it != s.end(); ++it) cout << *it;
```

Remember: char c is a copy; char& c refers to the actual character and can modify s.

4. End Operations

```
string s = "ABC";
s.push_back('D'); // ABCD
s.pop_back(); // ABC
```

Cost: push_back() amortized O(1), pop_back() O(1).

5. Append

```
s.append(" C++");
s += " C++";
```

Both add text to the end. += is concise; append() is explicit.

6. substr()

```
string s = "COMPUTER";
string x = s.substr(3, 4); // "PUTE"
```

Pattern: substr(start, length). The second argument is a **length**, not an ending index. Creating the result is typically O(length).

7. find()

```
size_t pos = s.find("world");
if (pos != string::npos) { /* found */ }
```

Returns the first matching starting position. Not found → string::npos. For practical DSA reasoning, treat a search as potentially linear; exact behavior depends on implementation/search case.

8. erase()

```
string s = "ABCDEFGHIIJ";  
s.erase(2, 3); // "ABFGHIIJ"
```

Pattern: erase(start, length). Removes 3 characters starting at index 2. Middle removal generally costs $O(n)$ because characters can shift.

9. insert()

```
string s = "ABCD";  
s.insert(2, "XYZ"); // "ABXYZCD"
```

Pattern: insert(position, text). Text is inserted **before** the character currently at that position. Middle insertion generally costs $O(n)$.

10. replace()

```
string s = "I like Java";  
s.replace(7, 4, "C++"); // "I like C++"
```

Pattern: replace(start, length, replacement) = WHERE + HOW MUCH + WITH WHAT. Middle replacement generally costs $O(n)$.

11. String Comparison

```
a == b  
a != b  
a < b  
a > b
```

Comparisons are **lexicographical**: compare left-to-right; the first differing character decides. If one is a prefix of the other, the shorter string is smaller. Comparisons are case-sensitive: "Apple" != "apple".

12. compare()

```
a.compare(b)
```

$< 0 \rightarrow$ a comes before b $== 0 \rightarrow$ equal $> 0 \rightarrow$ a comes after b.

Do not memorize the exact returned integer; the sign is what matters. Operators are often simpler in everyday DSA/CP.

13. Character Classification

```
isdigit(c) // digit?  
isalpha(c) // letter?  
islower(c) // lowercase?  
isupper(c) // uppercase?
```

A character can have type char while representing a digit, e.g. '7'. Manual digit check: $c \geq '0' \ \&\& \ c \leq '9'$.

14. Case Conversion

```
tolower(c) // returns lowercase version  
toupper(c) // returns uppercase version
```

They return a value; they do not automatically overwrite your variable.

```
char c = 'A';  
cout << tolower(c); // a, but c is still A  
c = tolower(c); // now c is a
```

Whole-string pattern:

```
for (char& c : s) c = tolower(c);
```

15. C-Style Strings vs std::string

C-style: char s[] = "HELLO"; is a character array with a terminating null character \0. Conceptually: H E L L O \0. The terminator tells C-style string functions where the text ends.

std::string: string s = "HELLO"; is a C++ object that manages its size and offers string member functions.

Important: char s[] = "CAT" needs storage for 4 chars (C A T \0), while string s = "CAT" has s.size() == 3.

16. DSA / Competitive Programming Pattern Recognition

When you see...	Think...
character at position i	s[i]
last character	s.back()
count a target character	traverse + if(c == target) count++
count digits	isdigit(c)
count letters	isalpha(c)
convert all characters to lowercase	for (char& c : s) c = tolower(c)
remove a section	erase(start, length)
insert text at position	insert(position, text)
replace a section	replace(start, length, text)
extract a section	substr(start, length)
search for text	find() + string::npos
alphabetical/content comparison	==, !=, <, >

17. Complexity Quick Reference

Operation	Typical / practical complexity
s[i], front(), back()	O(1)
size(), length(), empty()	O(1)
push_back()	Amortized O(1)
pop_back()	O(1)
Full traversal of n chars	O(n)
substr(start, len)	O(len) to produce the substring
Middle insert / erase / replace	Generally O(n)
find()	Potentially O(n) in ordinary use; exact behavior depends on implementation/search

18. Common Mistakes

- **Off-by-one:** last index is size()-1, not size().
- **substr:** second argument is length, not ending index.
- **erase:** erase(start, length), not erase(start, endIndex).
- **insert:** inserts before the character currently at the position.
- **find:** failure is string::npos; don't rely on -1.
- **Case:** 'A' and 'a' are different.
- **Range-for:** char c modifies a copy; char& c can modify the string.
- **tolower/toupper:** assign the returned value back if you want to modify the string.
- **C-style:** remember the terminating \0.

19. Interview Checklist

1. Why use std::string instead of a raw char array?
2. Why is std::string not literally vector?
3. size() vs length()?
4. Why is s[s.size()] not a valid last-character access?
5. What is string::npos?
6. What is lexicographical comparison?
7. What does compare() communicate through its sign?
8. Why does char s[] = "HELLO" contain \0?
9. char vs char& in range-based loops?

10. Why are middle insert/erase operations generally $O(n)$?

20. 30-Second Revision

std::string = a C++ string object holding a sequence of characters. Use `s[i]` for access, `size()/length()` for length, `front()/back()` for ends, `push_back()/pop_back()` for end changes, `+=/append()` for adding text, `substr()` for extraction, `find()` for search, `erase()/insert()/replace()` for modification, and operators for lexicographical comparison. Use character helpers for classification/case conversion. Remember that C-style strings end with `\0`.

Mastery status: std::string CLOSED ■